



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506 / SE ZG506

Lectures No. 5 and No. 6

Evaluation



Evaluation Component	Name	Type	Weight	Duration
EC - 1	Quiz I	Online	5%	
	Assignment I	Online	15%	
	Assignment II	Online	15%	
EC - 2	Mid-term Exam	Closed book	30%	2 hours
EC - 3	End Semester Exam	Open book	35%	2.5 hours

Quiz 1: 22-Feb-25 till 24-Feb-25

Agenda



- Data Science Project with Demo
- DataOps
- Building a Cloud-native Data Pipeline and accessing the application using APIs
 - Tool: Prefect, Prefect Cloud



BITS Pilani
Pilani Campus



Data Science Project with Demo

Data Science Project



Steps:

1. Data Ingestion [get data from data source]
2. Data Pre-processing or Cleaning
3. Data Exploration - EDA (Exploratory Data Analysis]
4. Data Analysis [ML code]
5. Data Visualization [Python - Matplotlib, Seaborn]

Data Science Project



Covid Patient Discharge Prediction (Dataset: 2nd Wave April-2021 to June 2021)

Type of Project: Data Science and Machine Learning

Dataset size: 1233 patients suffering from Covid

Variables:

X: Age, Gender, Co_morbid, Admit Date, Discharge date, days of stay, covid_severity

Y: Discharge Type (Recovered, Expired)

Exploratory Data Analysis: Univariate, Bivariate, Multivariate

Models applied: Support Vector Machine, Naïve Bayes, Logistic Regression, Decision Trees, KNN, ANN, Random Forest

Best Accuracy: Random Forest (92%)

Covid Dataset Overview



Gender: Categorical (M/F)

Age: Numeric

Co_Morbid: Binary (1 = Yes, 0 = No)

Admit_date & Discharge_date: Dates

Remdesivir_Given: Binary (1 = Given, 0 = Not Given)

DaysOfStay: Numeric

DischargeType: Numeric (1-Recovered; 2-Expired)

Covid_Severity: Numeric (1 = Mild, 2 = Moderate, 3 = Severe)

Covid_SeverityDescription: Categorical (Mild, Moderate, Severe)

DischargeTypeCategorical: Categorical (Recovered, Expired)

Demo



Gender: Categorical (M/F)

Age: Numeric

Co_Morbid: Binary (1 = Yes, 0 = No)

Admit_date & Discharge_date: Dates

Remdesivir_Given: Binary (1 = Given, 0 = Not Given)

DaysOfStay: Numeric

DischargeType: Numeric (1-Recovered; 2-Expired)

Covid_Severity: Numeric (1 = Mild, 2 = Moderate, 3 = Severe)

Covid_SeverityDescription: Categorical (Mild, Moderate, Severe)

DischargeTypeCategorical: Categorical (Recovered, Expired)



BITS Pilani
Pilani Campus



DataOps

Need for DataOps



- Can DevOps principles directly be applied to Data Science processes?
- What are the hindrances?
- Can data and code be treated the same way? For example, in DevOps, a working code is delivered to end-users at the end of CI/CD. Will a refined, pre-processed data be available for analysis to the Data Analysts or Data Scientists in the same agile way?

Data Challenges



- Adoption of DevOps to Data Science discipline is slow and gradual
- Data Science, and Data Exploration in particular **are more exploratory process**, compared to software development.
- **Bottlenecks**
 - Data Collection (more sources added or removed constantly)
 - Data Cleansing
 - Data Exploration (Univariate, Bivariate and Multivariate analysis)
 - Data Pipeline environments more difficult to reproduce compared to software application environments

DataOps



- It is a way to streamline, manage, and automate data flow within an Organization.
- Implementing DevOps strategies
- Includes Data Ingestion, transformation, processing, analysis
- Reduces time and resources required to manage the infrastructure

DataOps definition



Gartner Definition..

...a collaborative data management practice focused on improving the communication, integration and automation of data flows between data managers and consumers across an organization. The goal of DataOps is to create predictable delivery and change management of data, data models and related artifacts. DataOps uses technology to automate data delivery with the appropriate levels of security, quality and metadata to improve the use and value of data in a dynamic environment.⁵

DataOps definition



DataKitchen's Definition

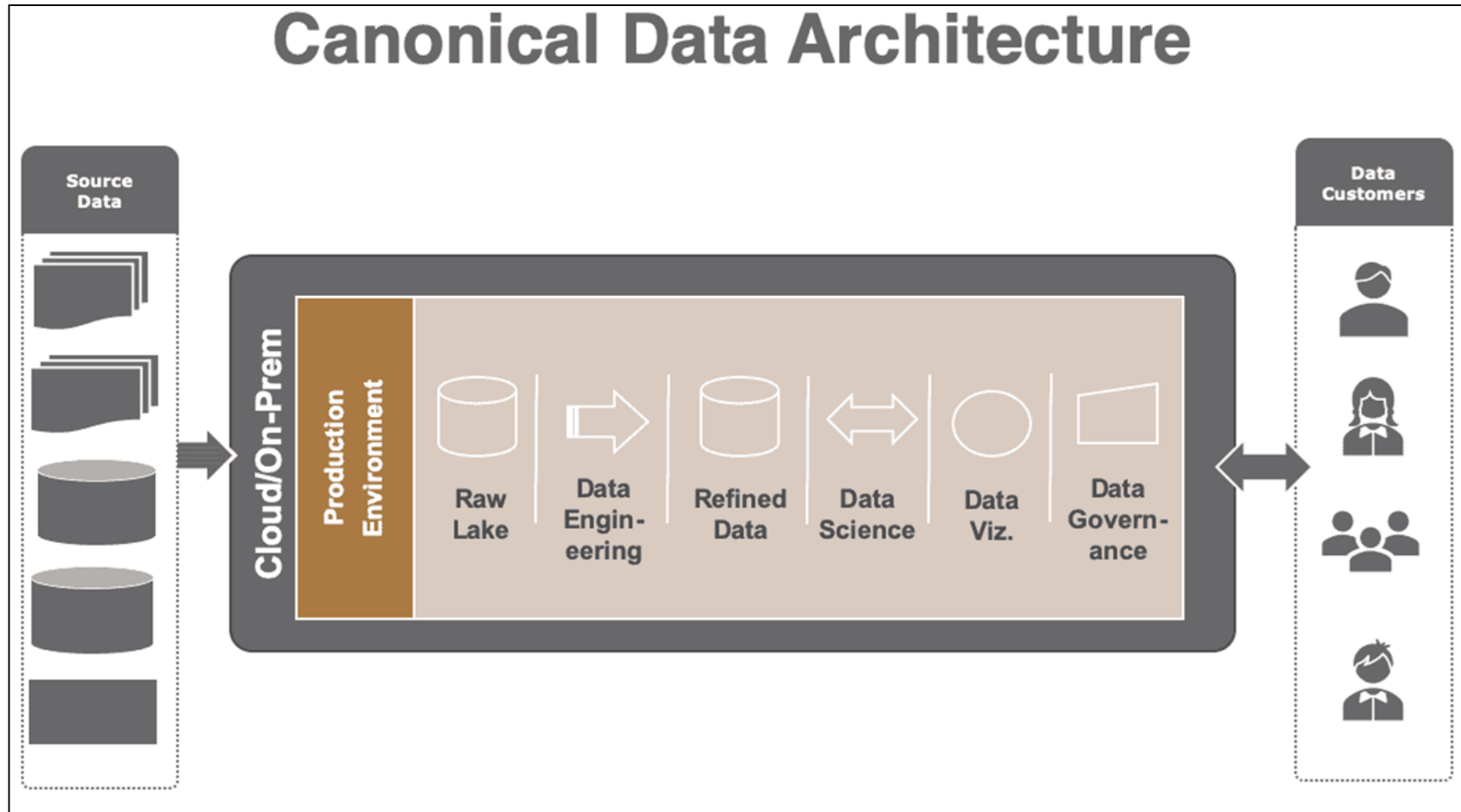
DataOps = Agile Practices + Lean Thinking + Data Analytics + DevOps

- **Agile Practices** ensure that we work on the “right things” that add value for the “right people”
- **Lean Thinking** focuses on eliminating waste and bottlenecks, improving quality, monitoring data flows, and making data cheaper for consumers
- **DevOps practices** build a culture of collaboration between historically siloed teams.
- **Data Analytics** - Descriptive, Diagnostic, Predictive and Prescriptive.
- Brings together self-contained teams having **Data Analysts, Data Scientists, Data Engineers, Machine Learning Engineers and DevOps Engineers.**

DevOps concepts used in DataOps

1. Source Code Management (Git)
2. Continuous Integration (CI)
3. Continuous Delivery / Deployment (CD)
4. Operations : Monitoring and Logging [Observability]
5. Culture: Collaboration and Communication

Canonical Data Architecture

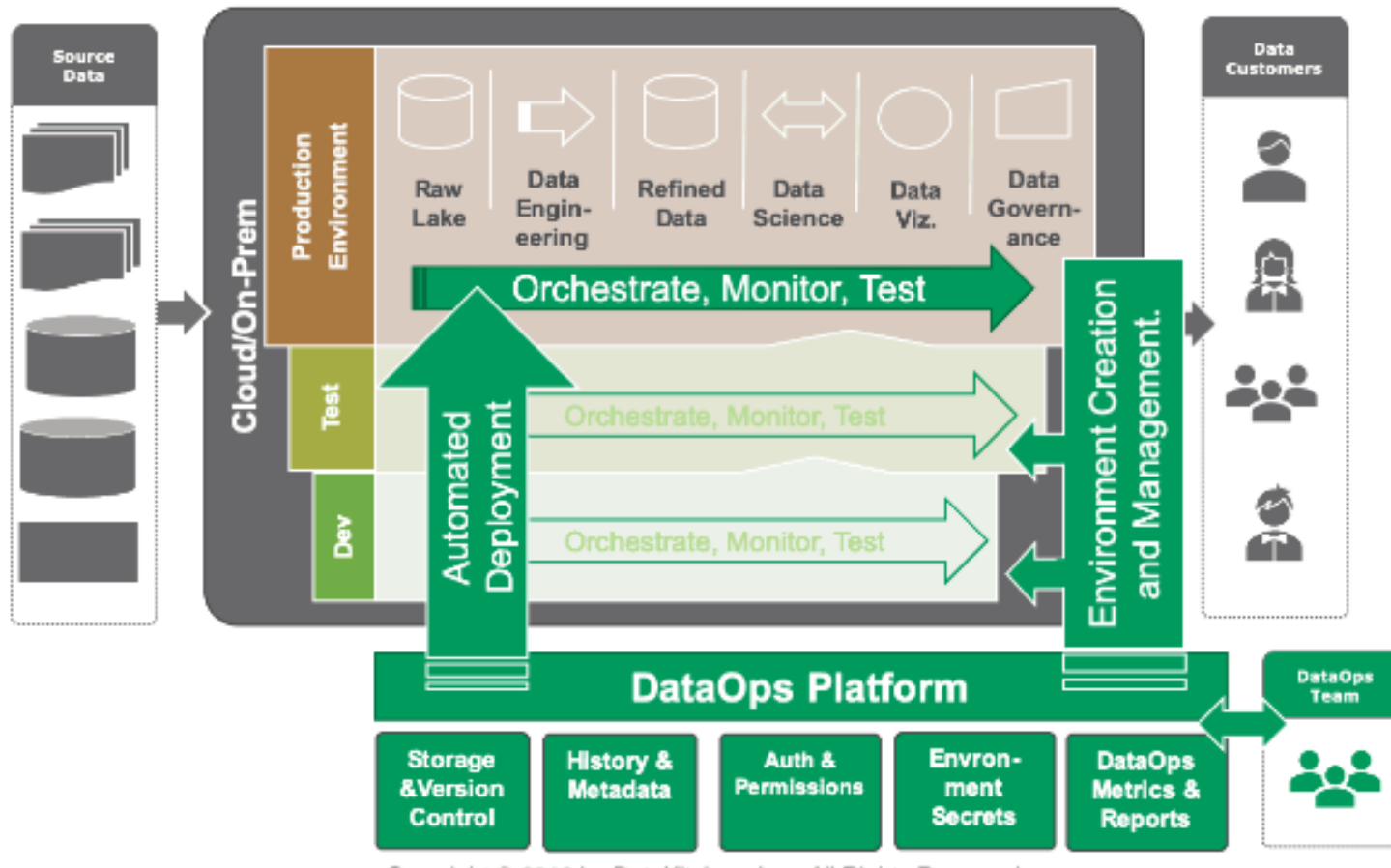


<https://datakitchen.io/dataops-data-architecture/>

DataOps Functional Architecture



DataOps Functional Architecture



DataOps Automation



1. Orchestrates, tests, and monitors the data pipeline.
2. Automate the process
 - Data flows in from hundreds or thousands of sources and is integrated, cleaned, processed, and published for analytics.
 - Check data for anomalies
 - For data errors, notify test results
 - Dashboard - View metrics based on monitoring

DataOps Tools



Open Source options

1. Apache AirFlow [Workflow automation tool]
2. Prefect [Workflow automation tool] and Prefect Cloud
3. DagSter [ETL pipelines and ML pipelines]
4. Curated list of DataOps tools - <https://github.com/kelvins/awesome-dataops>

Commercial options

1. DataKitchen [DataOps tool]
2. Unravel [Data Observability tool]



Building a Cloud-native Data Pipeline and Accessing using APIs



Prefect - Data Orchestration Tool

- Workflow orchestration tool for data scientists and ML engineers
- Only knowledge of Python is required, easy to learn [Pythonic]
- Open source
- Prefect Cloud is a cloud-native workflow engine
- REST API access for data management
- Transforms Python code into production-ready data pipelines
- Tools to build, monitor, and scale critical data workflows

URL - <https://www.prefect.io/>

Pre-requisites

1. `pip install -U prefect`

2a. Run Prefect on the Cloud [URL - <https://app.prefect.cloud/>]

`prefect cloud login` [Log in with a browser]

OR

2b. Run prefect locally -> `prefect server start`

URL - <https://docs.prefect.io/latest/tutorial/>

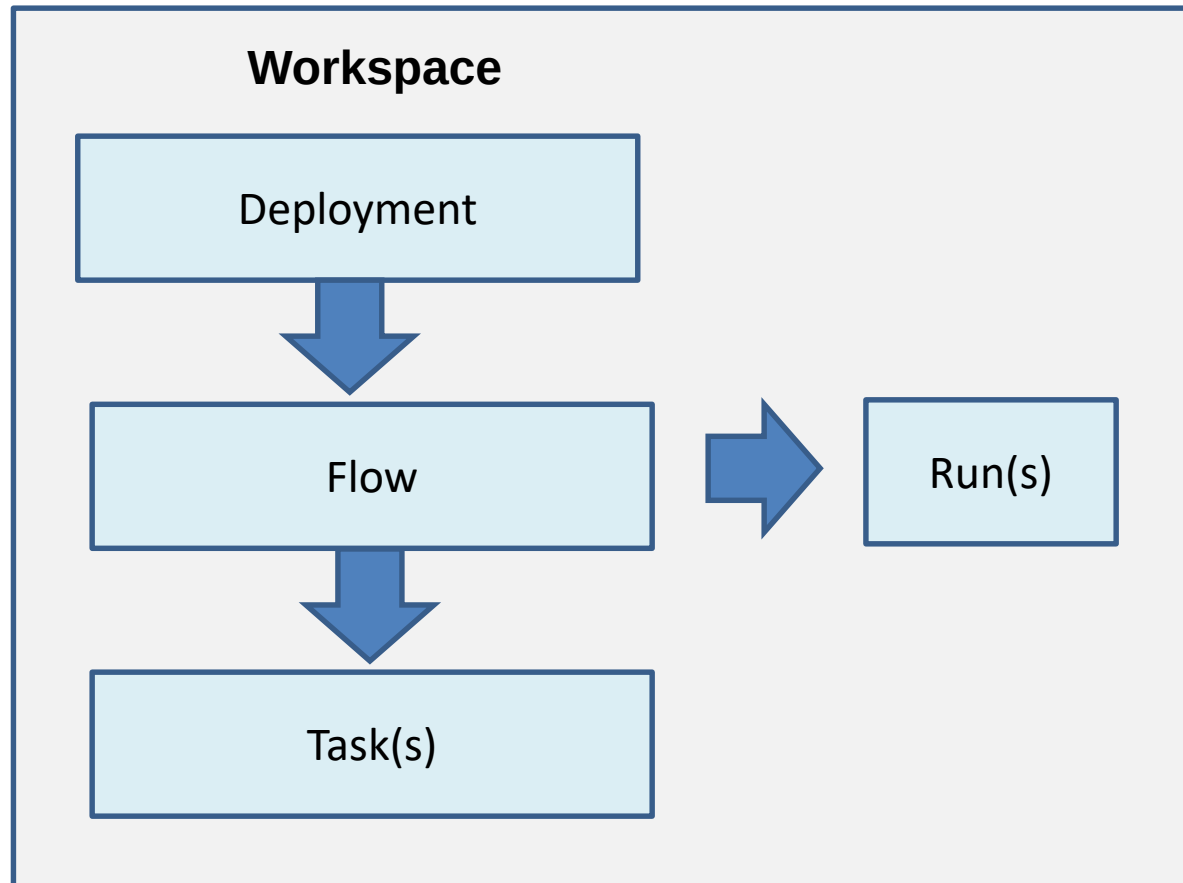
Prefect Architecture

Account ID
Workspace ID
Prefect API Key

Schedule or
trigger

Python
Functions with
annotators
@flow or @task

PREFECT CLOUD



Data Science Project



Steps:

1. Data Ingestion [get data from data source]
2. Data Pre-processing or Cleaning
3. Data Exploration - EDA (Exploratory Data Analysis]
4. Data Analysis [ML code]
5. Data Visualization

One Deployment

One Main Flow

Tasks for each
activity

Scheduled runs



BITS Pilani
Pilani Campus



Demo

Prefect Demo II



Minimal Example

Steps:

1. Import required library -> “Pima-Indian Diabetes dataset” [Classification]
2. Data Pre-processing
3. Model Training using Logistic Regression Model
4. Define the Prefect Flow and all the related tasks
5. Calculate the Accuracy and display in the “Prefect Cloud - Dashboard”

Data Science Project



Covid Patient Discharge Prediction (Dataset: 2nd Wave April-2021 to June 2021)

Type of Project: Data Science and Machine Learning

Dataset size: 1233 patients suffering from Covid

Variables:

X: Age, Gender, Co_morbid, Admit Date, Discharge date, days of stay, covid_severity

Y: Discharge Type (Recovered, Expired)

Exploratory Data Analysis: Univariate, Bivariate, Multivariate

Models applied: Support Vector Machine, Naïve Bayes, Logistic Regression, Decision Trees, KNN, ANN, Random Forest

Best Accuracy: Random Forest (92%)

Covid Dataset Overview



Gender: Categorical (M/F)

Age: Numeric

Co_Morbid: Binary (1 = Yes, 0 = No)

Admit_date & Discharge_date: Dates

Remdesivir_Given: Binary (1 = Given, 0 = Not Given)

DaysOfStay: Numeric

DischargeType: Numeric (1-Recovered; 2-Expired)

Covid_Severity: Numeric (1 = Mild, 2 = Moderate, 3 = Severe)

Covid_SeverityDescription: Categorical (Mild, Moderate, Severe)

DischargeTypeCategorical: Categorical (Recovered, Expired)

Prefect Demo II



Full Workflow demonstration for a Data Science Project

Executing the Project from VS Code terminal and running on Prefect Cloud.

Objectives: Observe the deployment, runs, flows and tasks in the Prefect Cloud Dashboard

<https://github.com/shreyassureshrao/dsp.git>

Folder Structure



```
project_directory/  
├─ data/  
│   └─ Covid_data.csv  
├─ flows/  
│   └─ workflow_covidex.py  
├─ tasks/  
│   ├─ BasicStats.py  
│   ├─ Binning.py  
│   ├─ ChiSquareCovidExample.py  
│   ├─ CorrelationCoeffecient.py  
│   ├─ Encoding.py  
│   ├─ FeatureImportanceMLAlgorithms.py  
│   ├─ Normalization.py  
│   ├─ PearsonCorrelation.py  
│   └─ Visualization.py  
└─ output/  
    └─ scatter_plot.png
```

Prefect APIs



- Prefect Cloud provides a REST API that allows you to interact with various entities, such as deployments, flow runs, task runs, etc.
- You can use this API to programmatically access data about your deployments and their runs.

Step 1: Get Prefect Cloud API Key

Step 2: Prefect Cloud REST API Access

<https://app.prefect.cloud/api/docs?deviceId=5349facc-c956-4511-869f-eeec8ad55a87>

Prefect APIs



Use the Prefect REST APIs, to get the following information:

1. Deployment details
2. Flow details

<https://github.com/shreyassureshrao/dsp.git>

Adding CI using GitHub Actions

- main.yaml file



```
name: Prefect CI/CD Pipeline with Prefect Cloud
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  pull_request:
```

```
    branches: [main]
```

```
jobs:
```

```
  run-prefect-workflows:
```

```
    runs-on: ubuntu-latest
```

```
  steps:
```

```
    - name: Check out repository
```

```
      uses: actions/checkout@v2
```

```
    - name: Set up Python 3.10
```

```
      uses: actions/setup-python@v2
```

```
      with:
```

```
        python-version: '3.10'
```

```
- name: Install dependencies
```

```
  run: |
```

```
    python -m pip install --upgrade pip
```

```
    pip install -r requirements.txt
```

```
- name: Authenticate with Prefect Cloud
```

```
  env:
```

```
    PREFECT_API_KEY: ${ secrets.PREFECT_API_KEY }
```

```
    WORKSPACE_KEY: ${ secrets.WORKSPACE_KEY }
```

```
  run: |
```

```
    prefect cloud login --key $PREFECT_API_KEY --workspace  
'shreyas-rao/default'
```

```
- name: Run Prefect deployment on Prefect Cloud
```

```
  env:
```

```
    WORKSPACE_KEY: ${ secrets.WORKSPACE_KEY }
```

```
  run: |
```

```
    prefect deployment run 'main-flow/covid-ds-workflow' --  
workspace-key $WORKSPACE_KEY
```

GitHub Actions

- GitHub's native CI/CD and automation system
- GitHub Actions was introduced by GitHub in 2018.
- It is integrated right into GitHub and enabled by default in every GitHub repository.

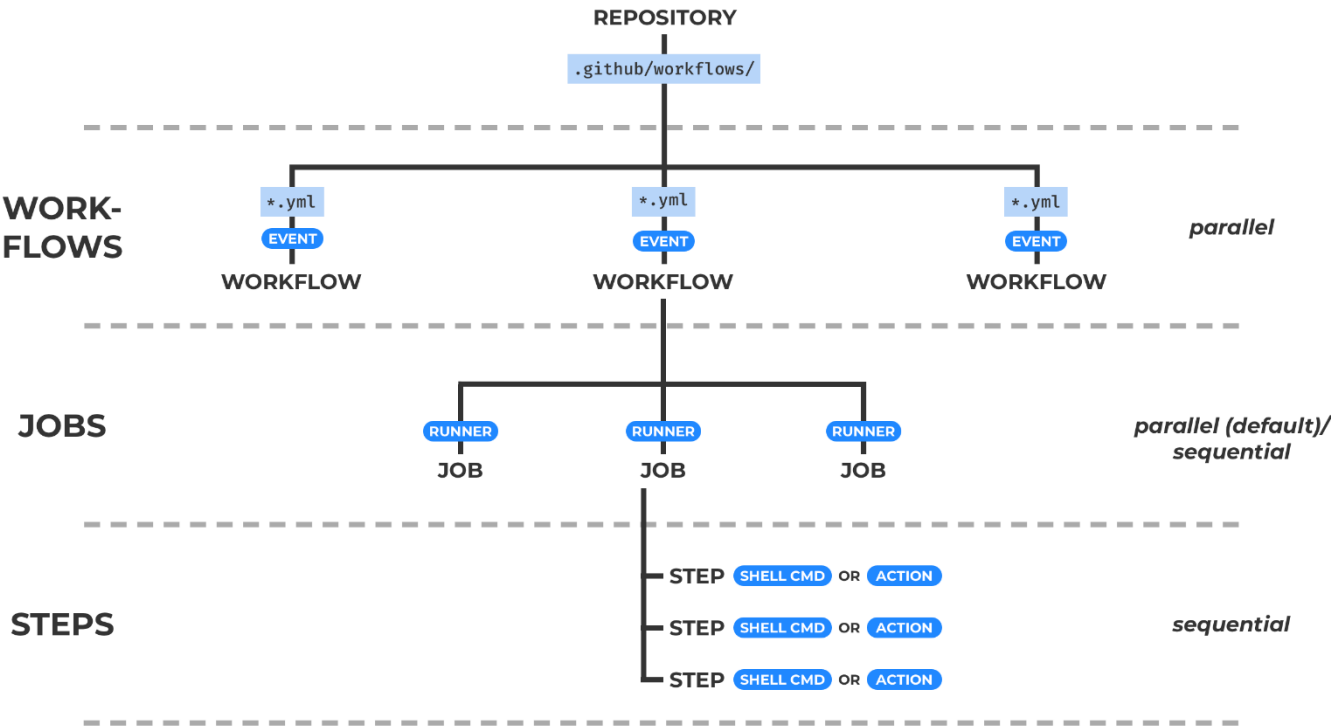
Typical GitHub Actions – Workflow structure

```
your-repository/
|-- .github/
|   |-- workflows/
|       |-- python-app.yml # Your GitHub Actions workflow file
|-- app/
|   |-- main.py
|-- tests/
|   |-- test_main.py
|-- requirements.txt
|-- Dockerfile
|-- README.md
|-- ... (other project files)
```

GitHub Actions



GitHub Actions



A workflow is triggered by one or more events. Event can be internal (push or PR) or external (webhook call)

A job contains a set of commands that will be run when the workflow is triggered

A runner is a virtual machine hosted by GitHub with an operating system. Each job runs inside a container

A step is either a shell command or an action

An action is a reusable unit of code which can be included as a step of a job. Actions can have inputs and outputs.



Thank You!